DESIGN AND ANALYSIS OF ALGORITHM

DYNAMIC PROGRAMMING

QUESTION 1:

Playing with Numbers:

Ram and Sita are playing with numbers by giving puzzles to each other. Now it was Ram term, so he gave Sita a positive integer ‘n’ and two numbers 1 and 3. He asked her to find the possible ways by which the number n can be represented using 1 and 3.Write any efficient algorithm to find the possible ways.

Example 1:

Input: *6*
Output:*6*
Explanation: *There are 6 ways to 6 represent number with 1 and 3*

1+1+1+1+1+1
3+3
1+1+1+3
1+1+3+1
1+3+1+1
3+1+1+1

Input Format
First Line contains the number n

Output Format

Print: The number of possible ways ‘n’ can be represented using 1 and 3

Sample Input

6

Sample Output

6

Answer:

```
#include <stdio.h>


int main() {
   int n;
   scanf("%d", &n);


   long long dp[n + 1];
```

```
    dp[0] = 1;


    for (int i = 1; i <= n; i++) {
        dp[i] = dp[i - 1];
        if (i >= 3) dp[i] += dp[i - 3];
    }


    printf("%lld\n", dp[n]);
    return 0;
}
```

RESULT:

Thus the code is verified and the expected output is obtained.

QUESTION 2:

Playing with Chessboard:

Ram is given with an n*n chessboard with each cell with a monetary value. Ram stands at the (0,0), that the position of the top left white rook. He is been given a task to reach the bottom right black rook position (n-1, n-1) constrained that he needs to reach the position by traveling the maximum monetary path under the condition that he can only travel one step right or one step down the board. Help ram to achieve it by providing an efficient DP algorithm.

Example:
Input
3
1 2 4
2 3 4
8 7 1
Output:
19

Explanation:
Totally there will be 6 paths among that the optimal is
Optimal path value:1+2+8+7+1=19

Input Format
First Line contains the integer n
The next n lines contain the n*n chessboard values

Output Format

Print Maximum monetary value of the path

Answer:

```
#include <stdio.h>


int main() {
   int n;
   scanf("%d", &n);


   int a[n][n];
   long long dp[n][n];


   for (int i = 0; i < n; i++)
      for (int j = 0; j < n; j++)
         scanf("%d", &a[i][j]);


   for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {
         if (i == 0 && j == 0) dp[i][j] = a[i][j];
         else if (i == 0)      dp[i][j] = dp[i][j - 1] + a[i][j];
         else if (j == 0)      dp[i][j] = dp[i - 1][j] + a[i][j];
         else {
            long long best = dp[i - 1][j] > dp[i][j - 1] ? dp[i - 1][j] : dp[i][j - 1];
            dp[i][j] = best + a[i][j];
         }
      }
   }
```

```
    printf("%lld\n", dp[n - 1][n - 1]);
    return 0;
}
```

RESULT:

Thus the code is verified and the expected output is obtained.

QUESTION 3:

Given two strings find the length of the common longest subsequence(need not be contiguous) between the two.

Example:

s1: ggtabe

s2: tgatasb

s1	**a**	**g**	***g***	***t***	***a***	***b***	
s2	***g***	**x**	***t***	**x**	***a***	**y**	***b***

The length is 4

Solveing it using Dynamic Programming

For example:

Input	Result
aab azb	2

Answer:

```
#include <stdio.h>
#include <string.h>

int main() {
```

```
    static char s1[10001], s2[10001];
    scanf("%10000s %10000s", s1, s2);

    int n = strlen(s1), m = strlen(s2);
    static int dp[2][10001];

    for (int i = 1; i <= n; i++) {
        int cur = i & 1, prev = cur ^ 1;
        for (int j = 1; j <= m; j++) {
            if (s1[i - 1] == s2[j - 1])
                dp[cur][j] = dp[prev][j - 1] + 1;
            else
                dp[cur][j] = dp[prev][j] > dp[cur][j - 1] ? dp[prev][j] : dp[cur][j - 1];
        }
    }

    printf("%d\n", dp[n & 1][m]);
    return 0;
}
```

RESULT:

Thus the code is verified and the expected output is obtained.

QUESTION 4:

Problem statement:

Find the length of the Longest Non-decreasing Subsequence in a given Sequence.

Eg:

Input:9

Sequence:[-1,3,4,5,2,2,2,2,3]

the subsequence is [-1,2,2,2,2,3]

Output:6

Answer:

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int a[n], tails[n], len = 0;

    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);

        int lo = 0, hi = len;
        while (lo < hi) {
            int mid = (lo + hi) / 2;
            if (tails[mid] <= a[i]) lo = mid + 1;
            else hi = mid;
        }
        tails[lo] = a[i];
        if (lo == len) len++;
    }

    printf("%d\n", len);
    return 0;
}
```

RESULT:

Thus the code is verified and the expected output is obtained.